

UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS



Construcción de Software I (Fundamentos, Prácticas)

Proyecto 4: Sistema de Gestión Académica

DOCENTE: Mag. Juan Luis Herencia Guerra

GRUPO: 4

- | | |
|-----------------------------------|-----------|
| • Chavez Miranda Leonardo Gabriel | 20240110J |
| • Ramos Jacay Dair David | 20232156D |
| • Eustaquio Avila Luis Gabriel | 20240206G |
| • Paucar Ventura Luis Antonio | 20240124K |
| • Chupa Ballesteros Gabriel | 20240160G |
-

2026-I

1. Introducción

1.1 Propósito

Este documento es la Guía de Despliegue y Operaciones del Sistema de Gestión Académica (SGA), su objetivo es describir, paso a paso, cómo preparar el entorno de desarrollo, configurar la base de datos en Supabase, publicar el backend en Render y el frontend en Vercel, y mantener el sistema funcionando correctamente.

Se trabaja de forma conjunta con el Plan de Implementación: si el plan define qué se va a construir y cómo está organizado el proyecto, esta guía explica cómo llevarlo a un entorno real y operativo.

1.2 Arquitectura de despliegue

El sistema adopta una arquitectura de tres niveles en producción, respetando la separación de capas definida en SRS-ARQ-SGA-2026:

Capa	Plataforma	Observación
Presentación	Vercel (SPA React)	Hosting estático con CDN global. Despliegue continuo desde rama main. Solo sirve assets estáticos; no ejecuta lógica de negocio.
Aplicación + Dominio	Render (Web Service)	Servidor ASGI persistente (Uvicorn). Ejecuta FastAPI con toda la lógica de negocio y validaciones. Expone la API REST consumida por el frontend.
Persistencia	Supabase (PostgreSQL 15)	Instancia PostgreSQL gestionada. Accedida exclusivamente desde Render via connection string. El frontend NO accede directamente a Supabase.

2. Prerrequisitos del Entorno

2.1 Herramientas requeridas

Herramienta	Versión mínima	Propósito
Python	3.12	Runtime del backend FastAPI.
Node.js	20 LTS	Runtime del frontend React / Vite.
Docker Desktop	26.x	Entorno de desarrollo local del backend. Garantiza reproducibilidad entre integrantes.
Git	2.x	Control de versiones. Conectado a repositorio GitHub del grupo.
Cuenta Supabase	Plan Free	Instancia de base de datos PostgreSQL gestionada.
Cuenta Render	Plan Free	Despliegue del backend FastAPI como Web Service.
Cuenta Vercel	Plan Hobby	Despliegue del frontend React.

3. Configuración del Entorno de Desarrollo Local

3.1 Clonar el repositorio

Ejecutar en terminal:

```
git clone https://github.com/[org]/sga-grupo4.git  
cd Proy_SW505_2026_1_Grupo_4
```

3.2 Variables de entorno

Tanto el backend como el frontend deberán poseer archivos .env con sus respectivas variables de entorno

3.3 Levantar el backend con Docker

Tendremos que construir la imagen de Docker de nuestro backend

3.4 Levantar el frontend en desarrollo

```
cd frontend  
npm install  
npm run dev
```

La aplicación React estará disponible en localhost con su puerto establecido.

4. Aprovisionamiento de la Base de Datos en Supabase

4.1 Crear el proyecto

Procedimiento:

1. Ingresar a <https://supabase.com> y crear un nuevo proyecto.
2. Seleccionar región más cercana.
3. Guardar la contraseña de la base de datos generada por Supabase. No se puede recuperar posteriormente.
4. Obtener la connection string desde Project Settings → Database → Connection String → URI (modo Transaction Pooler para producción, Direct Connection para desarrollo).

4.2 Aplicar el esquema físico

El esquema relacional completo (tablas, PKs, FKs, índices, constraints) está definido en el documento Modelo físico y implementado en el archivo backend/db/schema.sql del repositorio.

El script implementa la estrategia Multi-Tenant definida en la arquitectura: todas las tablas operativas incluyen la columna id_tenant con constraint de clave foránea hacia la tabla tenants y NOT NULL constraint. Ninguna operación puede leer o escribir datos de un Tenant distinto al del usuario autenticado.

4.4 Configuración de Row Level Security (RLS)

Supabase habilita Row Level Security por defecto en todas las tablas. Dado que el acceso a la base de datos se realiza exclusivamente desde el backend FastAPI mediante el rol service_role (que omite RLS), se deshabilita RLS a nivel de tabla y el aislamiento Multi-Tenant es gestionado por la capa de dominio en FastAPI.

Esto es coherente con el principio arquitectónico de que el aislamiento institucional es un invariante estructural verificado en la capa de dominio, no un mecanismo de base de datos.

5. Despliegue del Backend en Render

5.1 Justificación de la plataforma

Render se selecciona como plataforma de despliegue para el backend FastAPI por ser compatible con servidores ASGI persistentes (Uvicorn). Vercel no se utiliza para el backend porque su modelo de ejecución serverless impone un tiempo máximo de ejecución por request (10s en plan gratuito) y no mantiene conexiones de base de datos persistentes entre invocaciones, lo cual es incompatible con el pool de conexiones y el procesamiento asíncrono del cierre académico (RNFREN-05 establece hasta 10 minutos para el cierre de 2000 alumnos).

5.2 Procedimiento de despliegue

5. Conectar el repositorio de GitHub a Render.
6. Crear un nuevo Web Service apuntando al directorio backend/.
7. Configurar los parámetros del servicio:

Parámetro Render	Valor
Environment	Python 3
Build Command	pip install -r requirements.txt
Start Command	uvicorn app.main:app --host 0.0.0.0 --port \$PORT
Branch	main
Auto-Deploy	Yes — activado en cada push a main

8. Configurar las variables de entorno en Render (Environment → Environment Variables) con los mismos valores definidos en backend/.env, excepto ENVIRONMENT=production.
9. Verificar el despliegue accediendo al health check:

<https://backendDirce.onrender.com/health>

5.3 Consideraciones de operación en Render (plan gratuito)

Limitación	Impacto y mitigación
Cold start: el servicio se suspende tras 15 minutos de inactividad	Primer request tras inactividad tarda 30–60 segundos. Aceptable en contexto académico. Para el E4 (exposición final), hacer un request manual de activación 2 minutos antes.
750 horas/mes de ejecución en plan gratuito	Suficiente para el ciclo del proyecto. Monitorear en Render Dashboard.
No hay auto-scaling en plan gratuito	Suficiente para el volumen esperado en contexto académico (5 integrantes + docente evaluador).

6. Despliegue del Frontend en Vercel

6.1 Procedimiento de despliegue

10. Importar el repositorio de GitHub en Vercel.
11. Configurar el proyecto:

Parámetro Vercel	Valor
Framework Preset	Vite
Root Directory	frontend/
Build Command	npm run build
Output Directory	dist
Branch	main

12. Configurar la variable de entorno en Vercel (Project Settings → Environment Variables):

VITE_API_BASE_URL = https://[nombre-servicio].onrender.com/api/v1

13. Verificar que el build completa sin errores y que la aplicación carga correctamente en la URL asignada por Vercel.

6.2 Configuración de CORS

El backend FastAPI debe tener configurado CORS para aceptar requests desde el dominio de Vercel. En producción, la variable `ALLOWED_ORIGINS` debe incluir la URL exacta asignada por Vercel (formato: `https://[nombre-proyecto].vercel.app`).

7. Flujo de Despliegue Continuo (CI/CD)

7.1 Pipeline

#	Evento	Acción automática
1	Push a rama feature/*	GitHub ejecuta los tests (pytest) via GitHub Actions. Si los tests fallan, el PR no puede fusionarse.
2	Pull Request aprobado → merge a main	Render detecta el push a main y ejecuta el build (pip install + uvicorn start). Vercel detecta el push a main y ejecuta el build (npm run build + despliegue de dist/).
3	Build exitoso	Render actualiza el Web Service. Vercel actualiza el deployment. La nueva versión queda activa en producción.
4	Build fallido	Render y Vercel mantienen la versión anterior activa. Se notifica al equipo via email. No hay downtime.

7.2 Procedimiento de rollback

En caso de que un despliegue en producción introduzca un error crítico:

14. En Render: acceder al Dashboard → Deploys → seleccionar el deployment anterior → Rollback to this deploy.
15. En Vercel: acceder al Dashboard → Deployments → seleccionar el deployment anterior → Promote to Production.
16. Si el error es de base de datos: no ejecutar ningún script hasta identificar el problema. Supabase mantiene backups automáticos con RPO de 24 horas en plan gratuito.

8. Verificación Post-Despliegue

8.1 Checklist de verificación

#	Verificación	Resultado esperado
1	GET /health en Render	{ "status": "ok" } con HTTP 200
2	GET /docs en Render (Swagger UI)	Interfaz Swagger carga con todos los endpoints documentados
3	POST /api/v1/auth/login con credenciales del Administrador Central	Token JWT devuelto con rol y tenant correctos
4	Acceso al frontend en Vercel con credenciales de Alumno, Docente y Administrador	Vistas correctas por rol, sin errores de consola
5	Request desde frontend hacia backend (listar períodos académicos)	Respuesta correcta. CORS sin errores.
6	Variables de entorno en Render y Vercel no exponen valores sensibles en logs ni responses	No aparece JWT_SECRET_KEY ni DATABASE_URL en ninguna respuesta de la API

9. Monitoreo y Operaciones

9.1 Herramientas disponibles

Herramienta	Plataforma	Uso
Render Dashboard — Logs	Render	Logs de acceso y errores del backend en tiempo real. Primer punto de diagnóstico ante fallas.
Render Dashboard — Metrics	Render	CPU, memoria y tiempo de respuesta del Web Service.
Supabase Dashboard — Logs	Supabase	Logs de queries SQL. Útil para diagnosticar consultas lentas o errores de constraint.
Supabase Dashboard — Database	Supabase	Estado de conexiones activas, tamaño de tablas, índices.
Vercel Dashboard — Functions	Vercel	Logs de build y errores de despliegue del frontend.
Tabla auditoria_eventos en BD	Supabase	Registro inmutable de todas las operaciones del sistema con actor, timestamp y resultado (RNF-AUD-01). Consultar directamente via SQL Editor de Supabase.